

Nombre:

Fecha:

Profesor:

Materia:

Institución:

Curso:

Nota:

1. Análisis comparativo de patrones de diseño

El artículo destaca la importancia de los patrones de diseño en el desarrollo de software, pues permiten resolver problemas específicos y aportar extensibilidad y escalabilidad al sistema. Se presentan distintos patrones como *Template Method*, *MVC*, *MVP*, *Front Controller* y *MVVM*, explicando sus beneficios en la reducción de tiempos y mejora de eficiencia. Posteriormente, los autores realizan un análisis comparativo basado en artículos de investigación para identificar ventajas y desventajas de cada uno.

Finalmente, concluyen que la elección del patrón depende del tipo de proyecto, la experiencia del equipo y el equilibrio entre mantenibilidad, rendimiento y complejidad.

Además de esto, los patrones de diseño son considerados una herramienta fundamental en la ingeniería de software moderna, los autores rescatan que conocer y dominar diversos patrones proporciona a los desarrolladores una base sólida de buenas prácticas, que favorece la colaboración y el entendimiento común dentro del equipo de trabajo.

2. Revisión Sistemática de la Metodología para el Desarrollo de Software

El artículo presenta un análisis de la metodología **Scrum**, un marco de trabajo ágil orientado a la gestión de proyectos, especialmente en el desarrollo de Software. Se resalta su enfoque iterativo, colaborativo y adaptable a cambios, además de su origen en la década de **1980** y las diferentes perspectivas aportadas por diversas investigaciones.

Los autores aplican una revisión sistemática basada en artículos y bases de datos especializadas para evaluar su aplicación en este ámbito. Concluyen que Scrum proporciona un entorno flexible y controlado, fomenta el trabajo en equipo y facilita la resolución conjunta de problemas. No obstante, advierten que resulta más adecuado en proyectos con menor volumen de información y que su éxito depende de la correcta conformación de los equipos, evitando interferencias que comprometan la integridad del proyecto.

3. Perfiles del ciclo de vida del Software para pequeñas empresas: los nuevos estándares ISO/IEC 29110

El artículo analiza el papel crucial de la pyme en el desarrollo de software, destacando que la mayoría de las soluciones tecnológicas provienen de este tipo de empresa. Sin embargo, al enfrentarse a limitaciones de recursos humanos, económicos y organizativos, requieren prácticas de Ingeniería de Software eficientes y adaptadas a su contexto. Para entender esta necesidad, la ISO creó el grupo de trabajo SC7-WG24, cuyo objetivo es elaborar un estándar internacional que facilite la mejora de procesos de software específicamente en pymes. Este grupo busca establecer un marco común de perfiles evaluables del ciclo de vida de software, basados en la norma ISO/IEC 29110, lo que permitirá a estas empresas implementar procesos de calidad sin los altos costos de modelos tradicionales como CMMI o ISO 9001.

Al final del artículo se resalta que la iniciativa de ISO representa una oportunidad para que las pymes alcancen mejores niveles de productividad y competitividad en un entorno globalizado y cambiante. No obstante, también se reconocen las barreras que han dificultado la adopción de modelos más complejos en este tipo de organizaciones.

4. Arquitectura de Microservicios

El artículo introduce el concepto de microservicios, un enfoque de diseño de Software que consiste en dividir una aplicación en pequeños servicios independientes, cada uno encargado de una funcionalidad específica y comunicándose entre sí mediante interfaces ligeras. Este modelo contrasta con las arquitecturas monolíticas tradicionales, ya que busca mejorar la escalabilidad, flexibilidad y mantenibilidad de las aplicaciones, respondiendo a las crecientes demandas de los sistemas modernos. El autor plantea un ejemplo comparativo para ilustrar la diferencia entre ambas arquitecturas. En un sistema monolítico, a medida que la aplicación crece, el código se vuelve más complejo y difícil de mantener. En cambio, con una arquitectura de microservicios, cada módulo o servicio funciona de manera independiente. Además, se destaca que los microservicios facilitan la integración de nuevas tecnologías, permiten una mayor especialización de equipos de trabajo y ofrecen la posibilidad de desplegar y actualizar funcionalidades sin afectar el resto del sistema. Finalmente, el autor invita a reflexionar sobre cómo esta arquitectura brinda soluciones adaptadas a las necesidades actuales del desarrollo de software, dividiendo un problema complejo en subsistemas más pequeños e independientes.

5. Consideraciones sobre el desarrollo en infraestructuras críticas y software de Seguridad

El artículo aborda la creciente dependencia de las infraestructuras críticas del software, lo que incrementa la exposición a riesgos que pueden afectar tanto al mundo digital como al físico. En este contexto, se enfatiza la necesidad de garantizar la seguridad y calidad de los sistemas, en especial de aquellas aplicaciones consideradas críticas para el funcionamiento de una organización o sector. Se analiza el papel de los Sistemas de Gestión de Seguridad de la Información (SGSI), regulados por las normas ISO 27001, como herramientas clave para reducir amenazas y asegurar la continuidad operativa. Diversos estudios citados coinciden en que el mayor riesgo de seguridad no proviene de agentes externos, sino de errores internos, malas prácticas adoptadas dentro de las propias organizaciones.

Finalmente, el artículo concluye que el gran reto consiste en garantizar que el desarrollo de Software crítico cumpla con altos estándares de calidad y seguridad, asegurado su propia resiliencia de los sistemas ante posibles amenazas.

6. Análisis comparativo entre frameworks, para el desarrollo de aplicaciones móviles multiplataforma

Esta tesis analiza la evolución de JavaScript desde sus inicios como Mocha en 1996 hasta consolidarse como la base de la programación en la web. A partir de ello, se resalta cómo TypeScript surge como una alternativa robusta al código tipado estático y mejor manejo de datos, facilitando la creación de proyectos más seguros y escalables. La investigación se desarrolló con estudiantes de octavo semestre, durante un periodo de 6 semanas, empleando un enfoque práctico y progresivo. Se abordaron actividades que incluyeron la creación de servidores, el consumo de información en formato JSON y la aplicación de arquitecturas como MVC. En este contexto, se analizaron tres de los principales frameworks modernos: Angular, React y Vue. Los resultados indicaron que el framework Angular, a pesar de su curva de aprendizaje más extensa, proporciona una base sólida que permite comprender otros marcos con mayor facilidad. React destacó por su sencillez y compatibilidad directa con TypeScript y HTML, mientras que Vue resultó más intuitivo especialmente después de experiencias previas en programación.

El estudio concluye que los frameworks en TypeScript impulsan la productividad, calidad del código y preparación profesional, ofreciendo metodologías centradas en la práctica y la resolución de problemas.

7. Evaluación del rendimiento de los motores de bases de datos MySQL y Firebird

El artículo presenta una investigación orientada a evaluar el rendimiento de dos sistemas de gestión de bases de datos ampliamente utilizados: MySQL y Firebird, para ello, se diseñó e implementó una aplicación genérica de inventarios que permitió comparar el comportamiento de ambos motores en escenarios controlados. El proceso metodológico incluyó la creación de un algoritmo denominado *Robot*, cuya función principal fue poblar automáticamente las bases de datos con información generada de manera pseudorandom. Posteriormente, se establecieron casos de prueba que facilitaron la ejecución de transacciones y la medición de tiempos de respuesta. Los resultados mostraron que MySQL presentó mayor rapidez ejecutando las operaciones en menor tiempo que Firebird. No obstante, los autores señalan que ofrecer un juicio absoluto sobre qué motor resulta inviable, ya que cada sistema posee fortalezas y debilidades. Finalmente, se concluye que realizar este tipo de estudios con software libre aporta ventajas, al ser accesible para instituciones y desarrolladores, contribuyendo tanto al aprendizaje como la optimización de recursos en proyectos reales.

8. Frameworks basados en TypeScript para el desarrollo de aplicaciones web interactivas

Este artículo busca entender la evolución de las tecnologías web y la relevancia de **TypeScript** como lenguaje que amplía las capacidades de JavaScript en el desarrollo moderno. Se destacan los frameworks **Angular**, **React** y **Vue** como herramientas fundamentales para la creación de aplicaciones interactivas, dinámicas y con bajo tiempo de respuesta. Además se resalta la importancia de **WebSockets** en la comunicación entre cliente y servidor, ya que permiten conexiones persistentes y de baja latencia, esenciales para aplicaciones en tiempo real. Los resultados obtenidos en la implementación con estudiantes reflejan que **Angular** posee una curva de aprendizaje más compleja, aunque una vez dominado facilita la transición hacia otros (frameworks) frameworks.

De igual forma, se concluye que el uso de técnicas de normalización, así como de hojas de estilos **CSS**, **SASS** y **LESS**, contribuye a mejorar la estructura, calidad y escalabilidad del código.

En conclusión, la experiencia con frameworks basados en TypeScript favorece el desarrollo de competencias técnicas, incrementa la productividad y prepara a los estudiantes para integrarse en entornos profesionales de desarrollo web.

9. Arquitectura de Software con programación orientada a objeto

El artículo tiene como propósito describir la arquitectura de Software con POO, se resalta que este paradigma facilita el desarrollo y mantenimiento de programas de gran complejidad, al estructurar el software en clases y objetos que interactúan entre sí para generar sistemas más funcionales y reutilizables. Esta metodología se ha incorporado en múltiples lenguajes de programación, tales como Python, Java, PHP, C++, siendo C++ y Java los más utilizados en la actualidad. El enfoque orientado a objetos promueve la reutilización de código, la modularidad y la escalabilidad, características esenciales para enfrentar los retos de la ingeniería de software moderna. Asimismo, se evidencia cómo la evolución de la arquitectura de software, ha permitido a los profesionales del área abordar problemas complejos de forma más eficiente.

La programación orientada a objetos aplicada a la arquitectura de Software no solo ofrece un marco sólido para el diseño y construcción de sistemas, sino que también impulsa la innovación y la mejora continua en el desarrollo de Software.

10. Patrón Modelo-Vista-Controlador

El patrón MVC surge como una solución para simplificar y optimizar el desarrollo de aplicaciones, especialmente aquellas que requieren manejar múltiples representaciones y sincronización de datos. Su principal objetivo es dividir el Software en tres componentes bien definidos: el modelo, encargado de la gestión de datos y la lógica del negocio; la Vista, responsable de mostrar la información al usuario; y el controlador, que actúa como intermediario entre ambos, procesando las entradas y determinando las respuestas adecuadas. Esta separación estructural permite que cada componente pueda implementarse y modificarse de manera independiente, lo cual facilita la actualización, mantenimiento y escalabilidad de las aplicaciones. Gracias a ello el código resulta más comprensible, ordenado y predecible, evitando la mezcla de funcionalidades que generan errores y dificultades en el desarrollo.

Esto contribuye no solo a mejorar la productividad, sino también a garantizar una mayor calidad en el producto final. En síntesis, el patrón MVC representa un paradigma esencial para la ingeniería de Software moderna, ofreciendo eficiencia, claridad y robustez en el desarrollo de sistemas.